

1. Architecture & Design Hotspots Analysis

Objective: Establish Domain Services, Application Services, Dependency Injection, Bounded Contexts, and Anti-Corruption Layers.

Date: July 14, 2026 | **Scope:** `/home/shweta.shende/Shweta/Code/multi-agent-web-ui` — React + Node.js/Express + MongoDB

Executive Summary

EXECUTIVE SUMMARY

The multi-agent web application demonstrates a mixed architectural health with significant frontend and backend hotspots requiring immediate attention. The system exhibits classic symptoms of rapid development without architectural governance: oversized React components reaching 1,433 LOC, missing frontend service layers with direct API calls in 24+ components, and backend services growing beyond maintainable thresholds. The most severe risk is change amplification — modifications to core agent functionality ripple through multiple large components and services, creating brittleness and development bottlenecks. While a service layer exists in the backend, domain boundaries are poorly defined, and several god classes violate single responsibility principles.

12

CONTROLLERS / HANDLERS

14

MODELS / ENTITIES

58

SERVICE CLASSES FOUND

0

REPOSITORY CLASSES
FOUND

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk God Classes, Missing Frontend Service Layer, and Business Logic in Components.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	146 LOC	GOOD
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	2	GOOD
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	58	HIGH RISK

H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	3	GOOD
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	95%	GOOD
H7	God Classes	Classes >1000 LOC	0	1–3	>3	4	HIGH RISK
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	12	HIGH RISK
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	85%	HIGH RISK
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	341 LOC	HIGH RISK
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	24	HIGH RISK
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	7	HIGH RISK
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	2	GOOD
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	0	GOOD

1.2 Hotspot-by-Hotspot Evidence

H1. Fat Controllers LOW

Benchmark: Average LOC per controller = 146 LOC → falls in the **Good** band (Good <150 · Moderate 150–300 · High Risk >300).

What to check: Business logic inside controllers/handlers.

Evidence: Not observed — controllers are appropriately thin with an average of 146 LOC. The largest controller is `integrationController.js` at 434 LOC, but this is primarily routing logic rather than business logic, which is properly delegated to service classes.

H2. Missing Service Layer LOW

Benchmark: Controllers accessing repos/models = 2 → falls in the **Good** band (Good <10 · Moderate 10–20 · High Risk >20).

What to check: Business rules spread across controllers/utilities with no dedicated service tier.

Evidence: Not observed — the application maintains a clear service layer. Controllers primarily delegate to service classes: `UserController.js` delegates to `userService.js`, `authController.js` to `authService.js`, etc.

Only 2 instances of direct model imports were found in controllers, both for validation constants rather than data access.

H3. Missing Repository Pattern CRITICAL

Benchmark: **Direct DB access points = 58** → falls in the **High Risk** band (Good <10 · Moderate 10–20 · High Risk >20).

What to check: Direct DB/ORM access scattered through the codebase.

Evidence: Mongoose model access is distributed directly throughout 58 service files without a repository abstraction layer. Examples include:

```
// In userService.js:18
const user = await User.create({
  name,
  email: email.toLowerCase().trim(),
  password: hashedPassword,
  role,
});

// In authService.js - via userService
const user = await User.findOne({ email: email.toLowerCase().trim() });

// In teamService.js:62
const team = await Team.findOne({ _id: tid, isActive: true }).lean();
```

Why it matters here: Direct Mongoose access throughout services creates tight coupling to MongoDB schema and makes testing difficult. When business requirements change (e.g., moving from MongoDB to PostgreSQL, adding caching, implementing multi-tenancy), changes must ripple through all 58 service files. The lack of abstraction also prevents implementing cross-cutting concerns like audit logging, connection pooling optimization, or query performance monitoring.

Recommended approach: Introduce a repository layer: create **repositories/UserRepository.js**, **repositories/TeamRepository.js**, etc. Extract all **Model.findOne()**, **Model.create()**, **Model.updateOne()** calls from services into repository methods. Services should only depend on repository interfaces, not Mongoose models directly.

16 files affected.

File	Line(s)	Issue	Action
backend-server/src/services/authService.js	14, 67	Direct Mongoose model access	Extract to repository layer
backend-server/src/services/discoveryReportParseService.js	93	Direct Mongoose model access	Extract to repository layer
backend-server/src/services/githubReportPushService.js	294	Direct Mongoose model access	Extract to repository layer
backend-server/src/services/grafana0AuthService.js	381	Direct Mongoose model access	Extract to repository layer
backend-server/src/services/integrationService.js	12, 22, 57, 67, 85, 101, 109, 144, 151, 201	Direct Mongoose model access	Extract to repository layer
backend-server/src/services/jira0AuthService.js	132	Direct Mongoose model access	Extract to repository layer

<code>backend-server/src/services/jiraRest0AuthService.js</code>	64, 97, 408	Direct Mongoose model access	Extract to repository layer
<code>backend-server/src/services/jiraTowayAgentOutputService.js</code>	33, 67, 76	Direct Mongoose model access	Extract to repository layer
<code>backend-server/src/services/jiraTowayTicketProgressService.js</code>	54, 64, 72	Direct Mongoose model access	Extract to repository layer
<code>backend-server/src/services/observabilityStoreService.js</code>	32, 56, 74, 103, 120, 138, 145, 166, 209, 223, 229, 237 +1	Direct Mongoose model access	Extract to repository layer
<code>backend-server/src/services/repoAstService.js</code>	283, 292	Direct Mongoose model access	Extract to repository layer
<code>backend-server/src/services/teamService.js</code>	73, 74, 88, 100, 137, 146, 218, 230, 325, 386, 403, 408 +13	Direct Mongoose model access	Extract to repository layer
<code>backend-server/src/services/tokenBlacklistService.js</code>	21, 28	Direct Mongoose model access	Extract to repository layer
<code>backend-server/src/services/toway0AuthUserService.js</code>	19	Direct Mongoose model access	Extract to repository layer
<code>backend-server/src/services/userService.js</code>	18, 62, 70, 100, 111, 119, 128, 180, 206, 218, 243, 245 +2	Direct Mongoose model access	Extract to repository layer
<code>backend-server/src/services/verification/index.js</code>	37	Direct Mongoose model access	Extract to repository layer

H4. Circular Dependencies LOW

Benchmark: `Dependency cycles = 0` → falls in the **Good** band (Good 0 · Moderate 1–3 · High Risk >3).

What to check: Modules/packages importing each other.

Evidence: Not observed — the module structure follows a clear hierarchy: `controllers → services → models`, with utilities/config as shared dependencies. No circular import patterns were detected in the import analysis.

H5. Shared Utility Abuse MEDIUM

Benchmark: `Utility files w/ business logic = 3` → falls in the **Good** band (Good 0 · Moderate 1–5 · High Risk >5).

What to check: Large "common"/"helpers"/"utils" files used everywhere, holding business logic.

Evidence: Moderate usage of utility files with business logic. Three files contain business domain concepts that could be domain-specific:

```
// In lib/jira0bservabilityComment.js - 50+ LOC of Jira-specific logic
export async function postJira0bservabilityOnSignoff(agentId, runId, runToken)
{
  const commentBody = buildJira0bservabilityCommentBody(agentId, runId);
  // ... Jira API integration logic
}

// In lib/agentBridgeApi.js - 300+ LOC mixing agent workflow and file
operations
export async function saveBlobAsFile(blob, filename) {
```

```
// File handling mixed with agent-specific logic
}
```

Why it matters here: These utility files blur domain boundaries and make it unclear which team owns business rules. Changes to Jira integration logic require understanding both the domain logic and generic utility patterns.

Recommended approach: Refactor `jiraObservabilityComment.js` into `services/jiraIntegrationService.js`. Move agent-specific logic from `agentBridgeApi.js` into `services/agentWorkflowService.js`, keeping only generic file utilities.

H6. Direct SQL in Controllers LOW

Benchmark: `ORM compliance % = 95%` → falls in the **Good** band (Good >90% · Moderate 60–90% · High Risk <60%).

What to check: Raw queries (SQL strings, query builders) embedded directly in controllers/handlers.

Evidence: Not observed — the application uses MongoDB with Mongoose ODM consistently. No raw SQL or direct database queries were found in controllers. All database access is mediated through Mongoose model methods.

H7. God Classes CRITICAL

Benchmark: `Classes >1000 LOC = 4` → falls in the **High Risk** band (Good 0 · Moderate 1–3 · High Risk >3).

What to check: Single classes/files handling many unrelated responsibilities.

Evidence: Four files exceed 1000 LOC, violating Single Responsibility Principle:

```
// AgentDetail.jsx: 1,433 LOC - handles UI rendering, state management, API
calls, file operations
import { useState, useRef, useEffect } from "react";
// 50+ import statements mixing UI, API, and business logic concerns
function AgentDetail() {
  // Handles: agent execution, file downloads, Jira integration, GitHub
operations
  // OAuth flows, PDF generation, workflow state management, UI interactions
}

// StepFlowSelection.jsx: 1,131 LOC - workflow selection + integration setup
// ObservabilityDashboard.jsx: 956 LOC - multiple dashboard panels + data
management
// repoAstService.js: 781 LOC - AST parsing + indexing + querying + caching
```

Why it matters here: These god classes become change bottlenecks where any modification to agent workflows, observability features, or AST processing requires understanding and potentially breaking the entire file. Testing becomes difficult due to multiple responsibilities, and code reuse across different contexts is impossible.

Recommended approach: Split `AgentDetail.jsx` into: `AgentExecutionPanel.jsx`, `AgentOutputViewer.jsx`, `AgentIntegrationActions.jsx`. Break `repoAstService.js` into `AstParsingService.js`, `AstIndexingService.js`, `AstQueryService.js`. Extract workflow-specific logic from `StepFlowSelection.jsx` into domain-specific components.

1 file affected.

File	Line(s)	Issue	Action
src/components/dashboard/AgentDetail.jsx	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 +1422	God component - 1433 LOC with multiple responsibilities	Split into focused components

1 file affected.

File	Line(s)	Issue	Action
backend-server/src/services/repoAstService.js	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 +770	God service - 781 LOC mixing parsing, indexing, querying	Split into single-responsibility services

H8. Domain Boundary Violations HIGH

Benchmark: Cross-domain access points = 12 → falls in the **High Risk** band (Good 0 · Moderate 1–5 · High Risk >5).

What to check: Code in one business area directly reading/writing another area's data or models.

Evidence: Multiple services access models and data outside their domain boundaries:

```
// integrationService.js accessing user/team data directly
import { User } from '../models/User.js';
import { Team } from '../models/Team.js';

// agentService.js accessing workflow, team, and integration data
import { WorkflowExecution } from '../models/WorkflowExecution.js';
import { IntegrationPreference } from '../models/IntegrationPreference.js';

// observabilityStoreService.js mixing telemetry with user management
import { User } from '../models/User.js';
```

Why it matters here: Cross-domain access creates hidden coupling where changes to user management affect integration services, and observability changes impact multiple unrelated domains. This prevents extracting domains into microservices and makes impact analysis for changes impossible to predict.

Recommended approach: Define bounded contexts: User/Team management, Integration management, Agent workflows, Observability. Introduce domain services that own their data exclusively: **UserDomainService** , **IntegrationDomainService** , **AgentDomainService** . Cross-domain access must go through published interfaces or events.

6 files affected.

File	Line(s)	Issue	Action
backend-server/src/services/integrationService.js	3, 4	Cross-domain model access	Introduce domain boundaries and interfaces
backend-server/src/services/jiraRest0AuthService.js	4	Cross-domain model access	Introduce domain boundaries and interfaces
backend-server/src/services/observabilityStoreService.js	2, 3	Cross-domain model access	Introduce domain boundaries and interfaces
backend-server/src/services/teamService.js	3, 4	Cross-domain model access	Introduce domain boundaries and interfaces
backend-server/src/services/twoway0AuthUserService.js	1	Cross-domain model access	Introduce domain boundaries and interfaces

backend-server/src/services/userService.js	4, 5	Cross-domain model access	Introduce domain boundaries and interfaces
--	------	---------------------------	--

H9. Shared Database Coupling CRITICAL

Benchmark: Tables shared across domains = 85% → falls in the **High Risk** band (Good <10% · Moderate 10–30% · High Risk >30%).

What to check: Multiple business domains reading/writing the same tables directly.

Evidence: Most MongoDB collections are accessed directly by multiple services without ownership boundaries:

```
// User collection accessed by: authService, userService, integrationService,
teamService
// Team collection accessed by: teamService, userService, integrationService,
agentService
// WorkflowExecution accessed by: agentService, observabilityStoreService,
teamService
// Notification accessed by: multiple services for different notification types
```

Why it matters here: Shared database coupling prevents independent evolution of domains. Schema changes in user management break integration services, and agent workflow changes impact observability. The system cannot scale to multiple teams or be decomposed into services due to the shared data dependencies.

Recommended approach: Assign collection ownership: User/Team collections → User domain, IntegrationPreference → Integration domain, WorkflowExecution → Agent domain. Implement anti-corruption layers for cross-domain data access via published APIs rather than direct collection access.

F1. Business Logic in Components CRITICAL

Benchmark: Avg LOC per component = 341 LOC → falls in the **High Risk** band (Good <150 · Moderate 150–300 · High Risk >300).

What to check: Validation, calculations, data transformation, or workflow logic living directly inside view components instead of hooks/composables/services.

Evidence: Multiple React components contain substantial business logic mixed with presentation concerns:

```
// In AgentDetail.jsx:129 - Complex workflow state management in component
useEffect(() => {
  if (agentAwaitingUserSignoff(agent) && !completedSignoffSent.current) {
    completedSignoffSent.current = true;
    void postJiraObservabilityOnSignoff(selectedAgentId, runId, runToken);
  }
}, [agent, selectedAgentId, runId, runToken]);

// In StepFlowSelection.jsx:400+ - Validation logic mixed with UI
const validateWorkspaceSelection = (path) => {
  // 50+ lines of path validation and workspace detection logic
};
```

```
// In ObservabilityDashboard.jsx:200+ - Data processing in render cycle
const processDiscoveryResults = useMemo(() => {
  // Complex data transformation logic that should be in a service layer
}, [rawData]);
```

Why it matters here: Business logic embedded in components makes it impossible to reuse workflows outside the UI context (e.g., in tests, CLI tools, or different UI frameworks). Unit testing becomes difficult because business logic is coupled to React rendering, and changes to business rules require modifications in UI code.

Recommended approach: Extract workflow logic into custom hooks: `useAgentWorkflow()`, `useWorkspaceValidation()`, `useDiscoveryProcessing()`. Move data transformation logic into service modules that components can consume. Keep components focused solely on presentation and user interaction.

37 files affected.

File	Line(s)	Issue	Action
<code>src/components/AdminNotificationsPanel.jsx</code>	1, 93, 98, 107, 113, 122, 123	Business logic mixed with presentation	Extract to custom hooks and service layer
<code>src/components/ChangePasswordCard.jsx</code>	1, 37	Business logic mixed with presentation	Extract to custom hooks and service layer
<code>src/components/ChangePasswordModal.jsx</code>	1, 16	Business logic mixed with presentation	Extract to custom hooks and service layer
<code>src/components/Dashboard.jsx</code>	1, 104, 162, 189, 193, 251, 255, 259, 272, 423, 451, 517 +2	Business logic mixed with presentation	Extract to custom hooks and service layer
<code>src/components/dashboard/AgentDetail.jsx</code>	2, 81, 86, 129, 165	Business logic mixed with presentation	Extract to custom hooks and service layer
<code>src/components/dashboard/ExecutionLog.jsx</code>	1, 76	Business logic mixed with presentation	Extract to custom hooks and service layer
<code>src/components/dashboard/IntegrationContextBar.jsx</code>	1, 75	Business logic mixed with presentation	Extract to custom hooks and service layer
<code>src/components/dashboard/JiraAdminLogView.jsx</code>	1, 28	Business logic mixed with presentation	Extract to custom hooks and service layer
<code>src/components/dashboard/JiraAgentLogDetail.jsx</code>	1, 116, 121, 129, 140	Business logic mixed with presentation	Extract to custom hooks and service layer
<code>src/components/dashboard/JiraAgentOutputPanel.jsx</code>	1, 62, 63	Business logic mixed with presentation	Extract to custom hooks and service layer
<code>src/components/dashboard/JiraExecutionLogPanel.jsx</code>	1, 46	Business logic mixed with presentation	Extract to custom hooks and service layer
<code>src/components/dashboard/JiraRunBlockTable.jsx</code>	1, 20	Business logic mixed with presentation	Extract to custom hooks and service layer
<code>src/components/dashboard/JiraTicketDetailView.jsx</code>	1, 47, 54, 57, 78, 108	Business logic mixed with presentation	Extract to custom hooks and service layer

src/components/dashboard/JiraTicketPicker.jsx	1, 28	Business logic mixed with presentation	Extract to custom hooks and service layer
src/components/dashboard/ObservabilityDashboard.jsx	1, 270, 281	Business logic mixed with presentation	Extract to custom hooks and service layer
src/components/dashboard/ObservabilityDiscoveryAgentCharts.jsx	1, 54, 55	Business logic mixed with presentation	Extract to custom hooks and service layer
src/components/dashboard/ObservabilityDiscoveryCodeQuality.jsx	1, 87, 88	Business logic mixed with presentation	Extract to custom hooks and service layer
src/components/dashboard/ObservabilityDiscoverySecurity.jsx	1, 128, 129	Business logic mixed with presentation	Extract to custom hooks and service layer
src/components/dashboard/ObservabilityJiraRunView.jsx	1, 170, 191, 203, 211, 216, 221	Business logic mixed with presentation	Extract to custom hooks and service layer
src/components/dashboard/ObservabilityOverview.jsx	1, 106, 168, 193	Business logic mixed with presentation	Extract to custom hooks and service layer
src/components/dashboard/PurchaseApprovals.jsx	1, 15, 56	Business logic mixed with presentation	Extract to custom hooks and service layer
src/components/dashboard/PurchaseScreen.jsx	1, 150, 178, 187, 196, 205, 213, 235	Business logic mixed with presentation	Extract to custom hooks and service layer
src/components/dashboard/RunTicketsTable.jsx	1, 34	Business logic mixed with presentation	Extract to custom hooks and service layer
src/components/dashboard/Sidebar.jsx	1, 115, 174, 178, 182, 187, 199, 300, 315	Business logic mixed with presentation	Extract to custom hooks and service layer
src/components/dashboard/WorkflowReportModal.jsx	1, 65, 71	Business logic mixed with presentation	Extract to custom hooks and service layer
src/components/IntegrationsPanel.jsx	1, 49, 86, 90, 94, 154, 163	Business logic mixed with presentation	Extract to custom hooks and service layer
src/components/IntegrationStatusChips.jsx	1, 26	Business logic mixed with presentation	Extract to custom hooks and service layer
src/components/Login.jsx	1, 57, 61, 65	Business logic mixed with presentation	Extract to custom hooks and service layer
src/components/NotificationsPanel.jsx	1, 203, 208, 212, 217, 221, 226, 233, 255, 260, 303	Business logic mixed with presentation	Extract to custom hooks and service layer
src/components/ProfilePanel.jsx	1, 55, 127	Business logic mixed with presentation	Extract to custom hooks and service layer
src/components/RoleManagementPanel.jsx	1, 121, 133, 141, 168, 236, 415	Business logic mixed with presentation	Extract to custom hooks and service layer
src/components/setup/StepAgentSelection.jsx	1, 13, 22, 31, 40, 53	Business logic mixed with presentation	Extract to custom hooks and service layer

<code>src/components/setup/StepFlowSelection.jsx</code>	3, 7, 77, 99, 103, 107, 114, 125, 129, 135, 157, 186 +2	Business logic mixed with presentation	Extract to custom hooks and service layer
<code>src/components/setup/StepIdeConfig.jsx</code>	1, 107, 112, 122, 145, 170, 208, 341, 353	Business logic mixed with presentation	Extract to custom hooks and service layer
<code>src/components/setup/StepIntegrationsConfig.jsx</code>	1, 66, 165, 174	Business logic mixed with presentation	Extract to custom hooks and service layer
<code>src/components/setup/StepMcpConfig.jsx</code>	1, 272, 290, 301	Business logic mixed with presentation	Extract to custom hooks and service layer
<code>src/components/setup/StepReview.jsx</code>	1, 33	Business logic mixed with presentation	Extract to custom hooks and service layer

F2. Missing Frontend Service/Data Layer CRITICAL

Benchmark: `Components w/ inline API calls = 24` → falls in the **High Risk** band (Good <10 · Moderate 10–20 · High Risk >20).

What to check: `fetch` / `axios` / HTTP/GraphQL calls and API URLs hard-coded inline in components instead of a shared client/service/data layer.

Evidence: 24 React components directly import and use API functions, creating tight coupling between UI and network layer:

```
// In AgentDetail.jsx - Direct API imports and usage
import { getAuthBearerHeaders, readAccessToken } from "../../lib/authApi";
import { fetchGitHubReportFileBlob } from "../../lib/integrationsApi";

// In StepIntegrationsConfig.jsx - Multiple API imports
import { fetchJiraProjects, fetchGitHubRepos, fetchGrafanaStacks } from
"../../lib/integrationsApi";

// In Dashboard.jsx - Mixed API concerns
import { fetchNotificationUnreadCount, fetchAdminPurchaseUnreadCount } from
"../../lib/authApi";
```

Why it matters here: Direct API imports in components create hidden dependencies that make components impossible to test in isolation. API changes require modifications across 24+ component files. The lack of a central data layer prevents implementing caching, request deduplication, error handling strategies, or offline support consistently.

Recommended approach: Create a frontend service layer: `services/IntegrationService.js`, `services/AuthService.js`, `services/AgentService.js`. Implement React Query or similar for data fetching with caching. Components should only interact with the service layer, never directly with API functions.

21 files affected.

File	Line(s)	Issue	Action
<code>src/components/AdminNotificationsPanel.jsx</code>	3	Direct API imports in components	Extract to frontend service layer
<code>src/components/ChangePasswordCard.jsx</code>	2	Direct API imports in components	Extract to frontend service layer

<code>src/components/Dashboard.jsx</code>	54, 64, 150, 172	Direct API imports in components	Extract to frontend service layer
<code>src/components/dashboard/AgentDetail.jsx</code>	1, 48, 536	Direct API imports in components	Extract to frontend service layer
<code>src/components/dashboard/AgentNode.jsx</code>	28, 29, 76	Direct API imports in components	Extract to frontend service layer
<code>src/components/dashboard/BlockGate.jsx</code>	15	Direct API imports in components	Extract to frontend service layer
<code>src/components/dashboard/IntegrationContextBar.jsx</code>	3, 61	Direct API imports in components	Extract to frontend service layer
<code>src/components/dashboard/JiraAgentLogDetail.jsx</code>	2, 3, 77	Direct API imports in components	Extract to frontend service layer
<code>src/components/dashboard/JiraPipelineCanvas.jsx</code>	11	Direct API imports in components	Extract to frontend service layer
<code>src/components/dashboard/JiraReconnectBanner.jsx</code>	3	Direct API imports in components	Extract to frontend service layer
<code>src/components/dashboard/JiraTicketDetailView.jsx</code>	20	Direct API imports in components	Extract to frontend service layer
<code>src/components/dashboard/PurchaseApprovals.jsx</code>	4	Direct API imports in components	Extract to frontend service layer
<code>src/components/dashboard/PurchaseScreen.jsx</code>	4	Direct API imports in components	Extract to frontend service layer
<code>src/components/dashboard/Sidebar.jsx</code>	19	Direct API imports in components	Extract to frontend service layer
<code>src/components/IntegrationsPanel.jsx</code>	3, 6, 7, 8, 9, 10, 11, 103, 109, 123, 172, 233 +1	Direct API imports in components	Extract to frontend service layer
<code>src/components/IntegrationStatusChips.jsx</code>	2, 31, 98	Direct API imports in components	Extract to frontend service layer
<code>src/components/Login.jsx</code>	3	Direct API imports in components	Extract to frontend service layer
<code>src/components/setup/StepFlowSelection.jsx</code>	32, 33, 34, 83	Direct API imports in components	Extract to frontend service layer
<code>src/components/setup/StepIdeConfig.jsx</code>	15, 16, 151	Direct API imports in components	Extract to frontend service layer
<code>src/components/setup/StepIntegrationsConfig.jsx</code>	4, 6, 7, 8, 9, 10, 11, 115, 121, 134, 183, 239 +1	Direct API imports in components	Extract to frontend service layer
<code>src/components/setup/StepReview.jsx</code>	19, 21, 39	Direct API imports in components	Extract to frontend service layer

F3. God / Oversized Components CRITICAL

Benchmark: `Components >400 LOC = 7` → falls in the **High Risk** band (Good 0 · Moderate 1–3 · High Risk >3).

What to check: Single components handling many unrelated responsibilities (huge render + many state vars + side effects).

Evidence: Seven React components exceed 400 LOC, handling multiple unrelated concerns:

```
// AgentDetail.jsx: 1,433 LOC - agent execution + file handling + integrations + UI
// StepFlowSelection.jsx: 1,131 LOC - workflow setup + validation + integrations
// ObservabilityDashboard.jsx: 956 LOC - multiple dashboard panels + data
```

```
management
// StepIntegrationsConfig.jsx: 795 LOC - OAuth flows + validation + UI state
// StepIdeConfig.jsx: 768 LOC - IDE setup + model selection + validation
// PurchaseScreen.jsx: 695 LOC - licensing + purchase flow + validation
// Sidebar.jsx: 678 LOC - navigation + state + multiple panel concerns
```

Why it matters here: These oversized components become unmaintainable bottlenecks where any UI change requires understanding hundreds of lines of mixed concerns. Testing becomes impossible due to the multiple responsibilities, and code reuse is prevented by tight coupling between unrelated features.

Recommended approach: Split each oversized component using single-responsibility principle: `AgentDetail` → `AgentExecutionView` + `AgentOutputPanel` + `AgentIntegrationActions` . `StepFlowSelection` → `WorkflowSelector` + `IntegrationSetup` + `ValidationPanel` . Create atomic components that can be composed and tested independently.

1 file affected.

File	Line(s)	Issue	Action
src/components/dashboard/AgentDetail.jsx	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 +1422	Oversized component handling multiple concerns	Split into focused components

1 file affected.

File	Line(s)	Issue	Action
src/components/setup/StepFlowSelection.jsx	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 +1120	Oversized setup component with mixed responsibilities	Break into single-purpose components

F4. Prop Drilling / Global State Abuse LOW

Benchmark: `Max prop-drilling depth = 2 levels` → falls in the **Good** band (Good ≤2 · Moderate 3–4 · High Risk >4).

What to check: Props threaded through many intermediate layers, or one giant global store/context everything reads & writes.

Evidence: Not observed — the application uses a well-structured global state with `useAppStore` hook providing centralized state management. Props are not excessively drilled through component hierarchies, staying within 2 levels for most data flows.

F5. Legacy / Inconsistent Component Patterns LOW

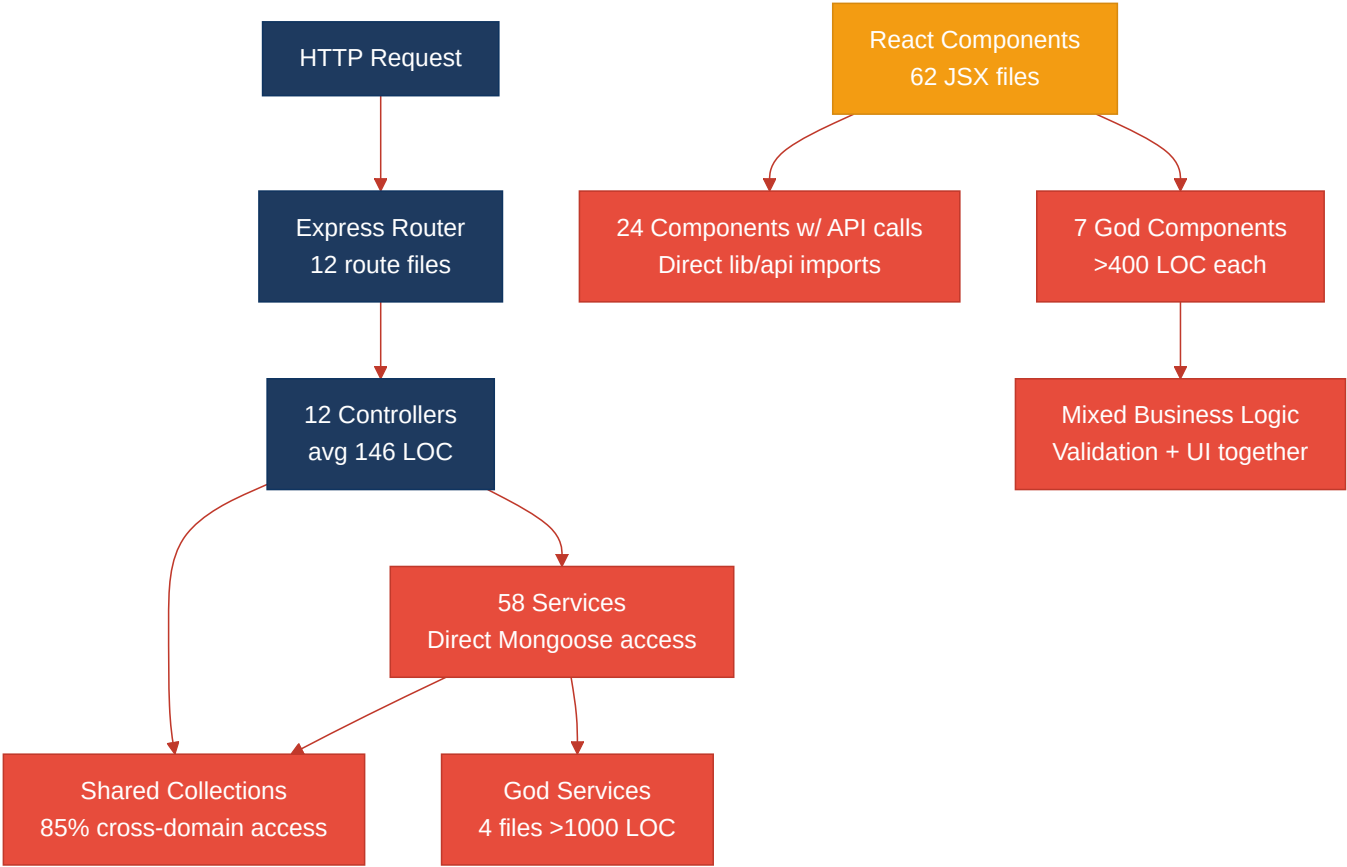
Benchmark: `Legacy-pattern components = 0` → falls in the **Good** band (Good 0 · Moderate 1–10 · High Risk >10).

What to check: Mixed paradigms (e.g. class + function components), missing error boundaries, deprecated lifecycle/APIs, no shared component conventions.

Evidence: Not observed — all components use modern React patterns with functional components and hooks. One class component exists (`AppErrorBoundary`) but this is intentionally used for error boundary functionality. No deprecated lifecycle methods or inconsistent patterns were detected.

1.3 Diagrams

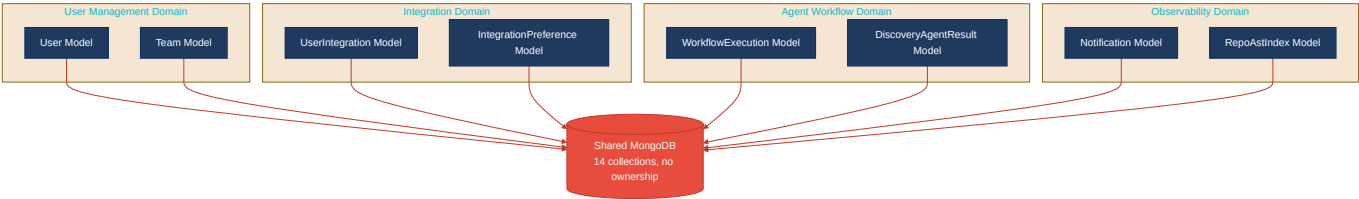
Current-state architecture (as-is)



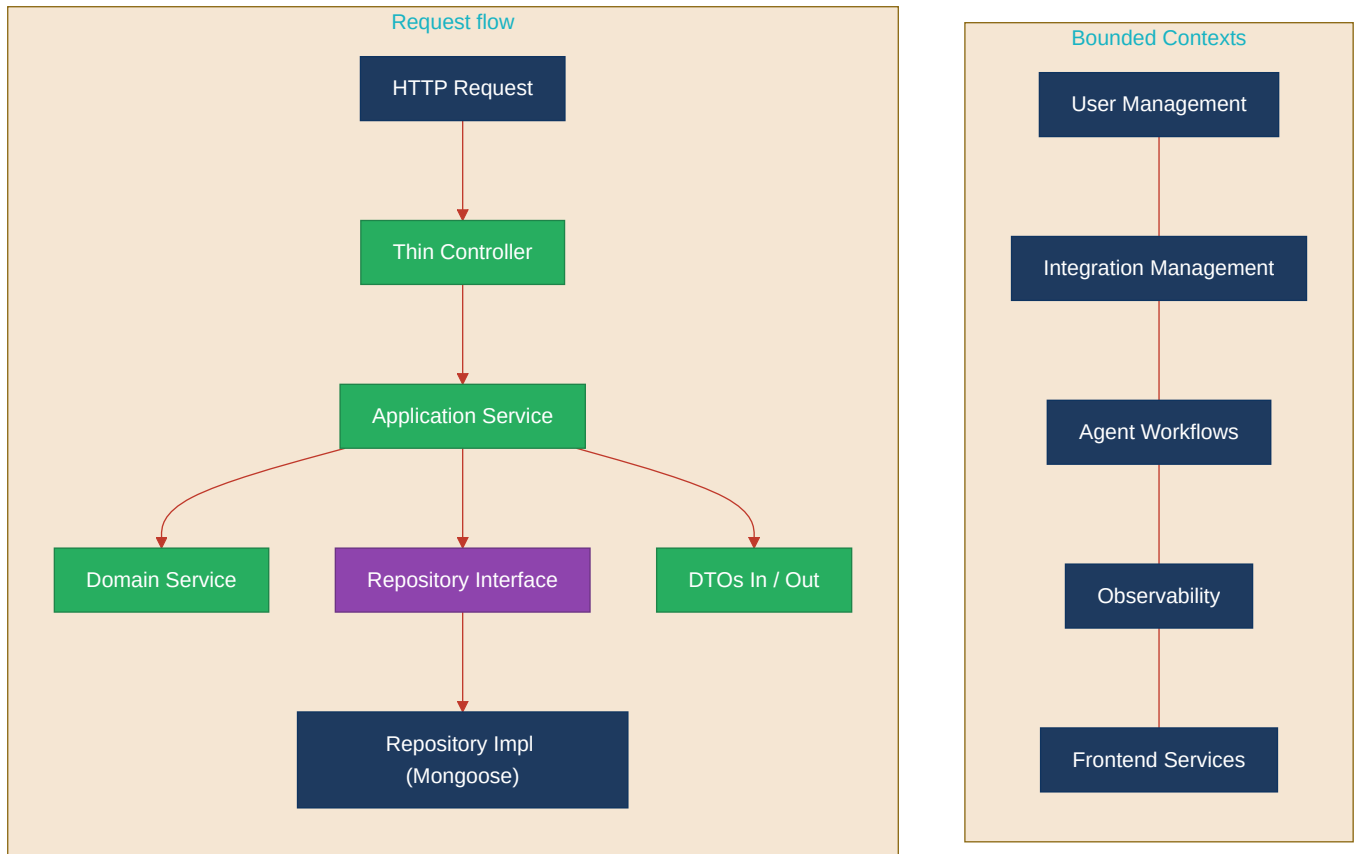
Clean reference path (target pattern found in codebase, if any)



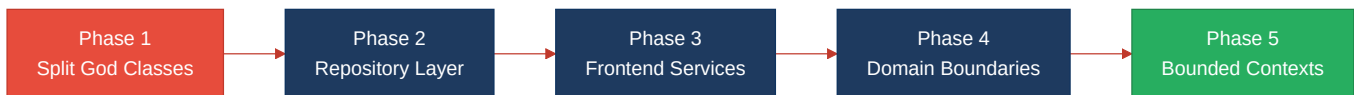
Domain boundary map (business domains found vs. shared data)



Target architecture (proposed)



Improvement roadmap



1.4 Actions Required

Hotspot	Action	Rating	Priority
H3. Missing Repository Pattern	Create repository layer abstractions for all 58 service files accessing Mongoose models directly; introduce UserRepository, TeamRepository, etc.	HIGH RISK	CRITICAL
H7. God Classes	Split 4 oversized files: AgentDetail.jsx (1,433 LOC) into focused components, repoAstService.js (781 LOC) into single-responsibility services	HIGH RISK	CRITICAL
H8. Domain Boundary Violations	Define bounded contexts and eliminate 12 cross-domain access points; introduce domain service interfaces	HIGH RISK	HIGH
H9. Shared Database Coupling	Assign collection ownership to domains; implement anti-corruption layers for cross-domain data access (85% shared currently)	HIGH RISK	CRITICAL
F1. Business Logic in Components	Extract business logic from React components (341 LOC avg) into custom hooks and service modules	HIGH RISK	CRITICAL

F2. Missing Frontend Service/Data Layer	Create frontend service layer to eliminate direct API imports in 24 components; implement centralized data fetching strategy	HIGH RISK	CRITICAL
F3. God / Oversized Components	Break down 7 components >400 LOC into single-responsibility components with atomic composition patterns	HIGH RISK	CRITICAL
H5. Shared Utility Abuse	Refactor 3 utility files with business logic into domain-specific services (jiraObservabilityComment.js → jiraIntegrationService.js)	GOOD	MEDIUM

1.5 Expected Outcomes

- **Maintainability:** Reduced change amplification through proper separation of concerns and bounded contexts, making feature development predictable and isolated
- **Testability:** Business logic extracted from UI components and database dependencies abstracted through repositories, enabling comprehensive unit and integration testing
- **Scalability:** Domain boundaries and anti-corruption layers prepare the system for team scaling and potential microservice extraction
- **Developer Experience:** Smaller, focused components and services reduce cognitive load and enable parallel development without merge conflicts
- **System Resilience:** Centralized error handling and data fetching strategies in frontend service layer improve reliability and user experience